

SETTLELATER

Credit and Ledger Management System

Technical Design & Architecture Documentation

Version: 1.0

Document Status: Technical Design

Date: 13 August 2026

Table of Contents

1. Introduction
 2. System Overview
 3. Core Domain Model
 4. Core Entities
 5. Credit Account
 6. Credit Management
 7. Ledger Transaction Management
 8. Financial Record Integrity and Auditability
 9. Credit Status Management
 10. Database Relationships and Constraints
 11. Credit Creation Workflow
 12. Payment Creation Workflow
 13. Concurrency Control
 14. REST API Design
 15. Request and Validation Design
 16. Balance Management Strategy
 17. Transaction Management
 18. Implementation Roadmap
 19. Final Architecture
 20. Design Principles and Future Extensibility
-

1. Introduction

1.1 Purpose

SettleLater is a digital credit and ledger management system designed to help shop owners manage customer credit transactions, repayments, outstanding balances, and transaction history in a structured and reliable manner.

The system replaces traditional notebook-based credit records with a centralized digital ledger. The architecture is designed to maintain accurate financial records while providing a clear transaction history for every customer and credit account.

A major architectural principle of SettleLater is that **financial records should be treated as ledger entries rather than ordinary editable records**. This approach improves data integrity, traceability, and auditability.

1.2 Core Objective

The primary objective of the financial module is to provide a reliable mechanism for:

- Creating and managing customer credit records.
 - Supporting multiple outstanding credits for a customer.
 - Recording partial and complete repayments.
 - Tracking outstanding balances.
 - Maintaining a complete transaction history.
 - Preventing invalid overpayments.
 - Maintaining an auditable financial history.
 - Supporting customers who belong to multiple shops.
 - Ensuring shop-level data isolation.
 - Handling concurrent payment requests safely.
-

2. System Overview

The core relationship within SettleLater is structured as follows:

User → Shop → Customer → Credit Account → Credit → Ledger Transaction

A shop owner owns one or more shops. Each shop manages its own customers. A customer may exist under multiple shops, but their financial relationship with each shop remains completely independent.

The financial model is based on a **Credit Account** that represents the relationship between one customer and one shop.

A single Credit Account can contain multiple individual credit entries.

Example

ABC Store

→ Ramesh
→ Credit Account
→ Credit #101 — ₹500
→ Credit #102 — ₹1,000
→ Credit #103 — ₹300

If the customer has made repayments of ₹200 against Credit #101 and ₹300 against Credit #103:

- Total Credits = ₹1,800
- Total Payments = ₹500

- Outstanding Balance = ₹1,300

This model allows the system to track individual credits while also maintaining an overall customer balance for a specific shop.

3. Core Domain Model

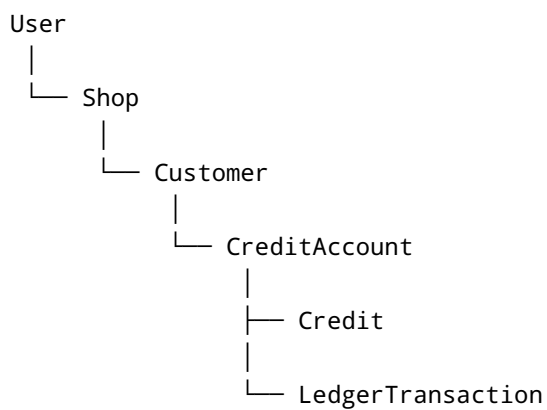
The financial domain consists of the following primary entities:

1. **Shop**
2. **Customer**
3. **CreditAccount**
4. **Credit**
5. **LedgerTransaction**

The existing system already contains the Shop and Customer entities. The financial module introduces three additional entities:

- CreditAccount
- Credit
- LedgerTransaction

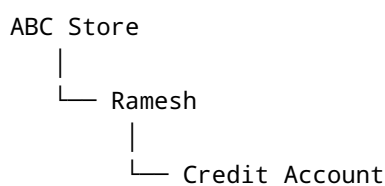
3.1 Domain Hierarchy

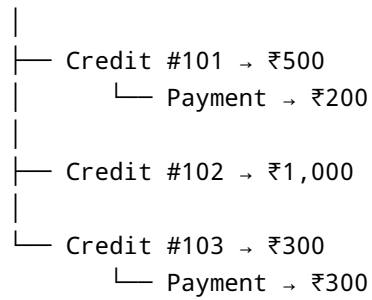


3.2 Multiple Credits per Customer

A customer may have multiple outstanding credits under the same shop.

For example:





The Credit Account therefore represents the overall relationship, while each Credit represents an individual financial obligation.

4. Core Entities

The financial module consists of the following entities.

4.1 Shop

The Shop represents a business owned by a registered shop owner.

A shop owns its customer relationships and financial records.

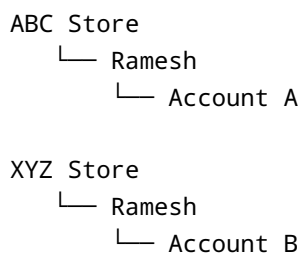
A shop owner must only be permitted to access and modify records belonging to their own shop.

4.2 Customer

A Customer represents an individual who purchases goods or services from a shop on credit.

A customer may belong to multiple shops.

For example:



Account A and Account B must remain completely independent.

4.3 CreditAccount

The CreditAccount represents the financial relationship between one customer and one shop.

Proposed attributes

```
CreditAccount
-----
id
shop_id
customer_id
total_credit
total_paid
outstanding_balance
status
created_at
updated_at
```

Relationship

```
Shop      1 ——— N CreditAccount
Customer  1 ——— N CreditAccount
```

The relationship allows the same customer to have separate financial accounts with different shops.

4.4 Credit

The Credit entity represents one individual credit transaction.

Proposed attributes

```
Credit
-----
id
credit_account_id
original_amount
outstanding_amount
credit_date
due_date
description
status
created_at
updated_at
```

Each Credit belongs to exactly one Credit Account.

A Credit may receive multiple payments throughout its lifecycle.

4.5 LedgerTransaction

LedgerTransaction represents an individual financial event associated with a credit account.

Proposed attributes

```
LedgerTransaction
-----
id
credit_account_id
credit_id
type
amount
transaction_date
description
reference_transaction_id
created_by
created_at
```

The `credit_id` should be nullable because future account-level adjustments may not necessarily belong to a specific credit.

For the initial MVP, CREDIT and PAYMENT transactions will normally be associated with a specific Credit.

5. Credit Account

5.1 Purpose

The CreditAccount provides an aggregated view of a customer's financial relationship with a particular shop.

It maintains:

- Total credit amount.
- Total amount paid.
- Current outstanding balance.
- Account status.
- Creation and update timestamps.

5.2 Account Uniqueness

A customer must not have multiple Credit Accounts for the same shop.

The database should therefore enforce:

```
UNIQUE(shop_id, customer_id)
```

This guarantees:

```
ABC Store + Ramesh → One Credit Account
```

while allowing:

```
ABC Store + Ramesh → Account A  
XYZ Store + Ramesh → Account B
```

This constraint is critical for maintaining consistent account-level balances.

6. Credit Management

6.1 Individual Credit

A Credit represents one specific amount owed by a customer.

Example:

```
Credit ID:      #101  
Customer:      Ramesh  
Amount:        ₹500  
Credit Date:   13-Aug-2026  
Due Date:      20-Aug-2026  
Description:    Grocery Items  
  
Original Amount: ₹500  
Outstanding:    ₹500  
Status:        UNSETTLED
```

6.2 Partial Payment

If the customer pays ₹200:

```
Original Amount: ₹500  
Paid:            ₹200  
Outstanding:     ₹300  
Status:          PARTIALLY_PAID
```

6.3 Full Settlement

If the customer subsequently pays ₹300:

```
Original Amount: ₹500
Outstanding:     ₹0
Status:          SETTLED
```

This allows the system to support multiple partial payments before a credit is fully settled.

6.4 Multiple Payments

A single credit can have multiple payment transactions.

Example:

```
Credit = ₹500

Payment 1 = ₹100
Payment 2 = ₹150
Payment 3 = ₹50
Payment 4 = ₹200
```

Total payment:

```
₹100 + ₹150 + ₹50 + ₹200 = ₹500
```

Outstanding balance:

```
₹500 - ₹500 = ₹0
```

The credit is therefore marked as:

```
SETTLED
```

7. Ledger Transaction Management

7.1 Purpose

LedgerTransaction is the most important component for maintaining financial history.

Instead of treating payments and credits as ordinary editable records, the system maintains a chronological financial ledger.

The ledger provides a complete history of what happened to an account.

7.2 Transaction Types

The supported transaction types are:

CREDIT
PAYMENT
ADJUSTMENT
REVERSAL

The MVP primarily uses:

CREDIT
PAYMENT

The additional transaction types provide a foundation for future financial corrections without destroying historical records.

7.3 Example Ledger

Suppose a customer receives a ₹500 credit.

The ledger initially contains:

ID	Type	Amount
1	CREDIT	₹500

The customer then pays ₹100:

ID	Type	Amount
1	CREDIT	₹500
2	PAYMENT	₹100

The customer subsequently pays ₹150, ₹50, and ₹200:

ID	Type	Amount
1	CREDIT	₹500

2	PAYMENT	₹100
3	PAYMENT	₹150
4	PAYMENT	₹50
5	PAYMENT	₹200

Total payments:

$$₹100 + ₹150 + ₹50 + ₹200 = ₹500$$

Outstanding:

$$₹500 - ₹500 = ₹0$$

Credit status:

SETTLED

8. Financial Record Integrity and Auditability

8.1 Immutable Financial History

Financial transactions should not be physically deleted once they have been recorded.

This principle ensures that the system can always answer:

What exactly happened to this customer's account?

Deleting financial records would create gaps in the transaction history and make it difficult to determine how the current balance was produced.

8.2 Payment Correction

Suppose the owner mistakenly records:

PAYMENT = ₹300

when the actual payment was ₹200.

The system should not simply delete the ₹300 transaction.

Instead, the transaction should be reversed.

The ledger becomes:

CREDIT	₹500
PAYMENT	₹300
REVERSAL	₹300
PAYMENT	₹200

The reversal neutralizes the incorrect payment while preserving the original record.

This approach provides complete auditability.

8.3 Credit Correction

The same principle applies to credit amounts.

Suppose the original credit was:

CREDIT	= ₹500
--------	--------

but the owner later realizes that the correct amount should have been ₹550.

The original transaction should not simply be modified from ₹500 to ₹550.

Instead:

CREDIT	₹500
ADJUSTMENT	₹50

The resulting financial history clearly explains why the account balance increased by ₹50.

If the adjustment itself is later determined to be incorrect:

REVERSAL	₹50
----------	-----

can be recorded.

8.4 Auditability

This design provides a complete and traceable financial history.

The ledger can therefore be used to determine:

- Which credit was created.
- When a payment was made.

- How much was paid.
- Which transaction was corrected.
- Why an adjustment occurred.
- How the current balance was derived.

This is a significant system-design advantage because financial history remains explainable and traceable.

9. Credit Status Management

The Credit entity supports the following statuses:

```
UNSETTLED  
PARTIALLY_PAID  
SETTLED  
OVERDUE
```

9.1 UNSETTLED

A credit is UNSETTLED when:

```
outstanding_amount > 0
```

and the credit has not yet reached an overdue state.

9.2 PARTIALLY_PAID

A credit is PARTIALLY_PAID when:

```
0 < outstanding_amount < original_amount
```

9.3 SETTLED

A credit is SETTLED when:

```
outstanding_amount = 0
```

9.4 OVERDUE

OVERDUE should be derived from the credit's outstanding amount and due date rather than being treated as an independent source of truth.

For example:

```
Outstanding = ₹500
Due Date    = 10-Aug-2026
Today       = 13-Aug-2026
```

The credit is overdue because:

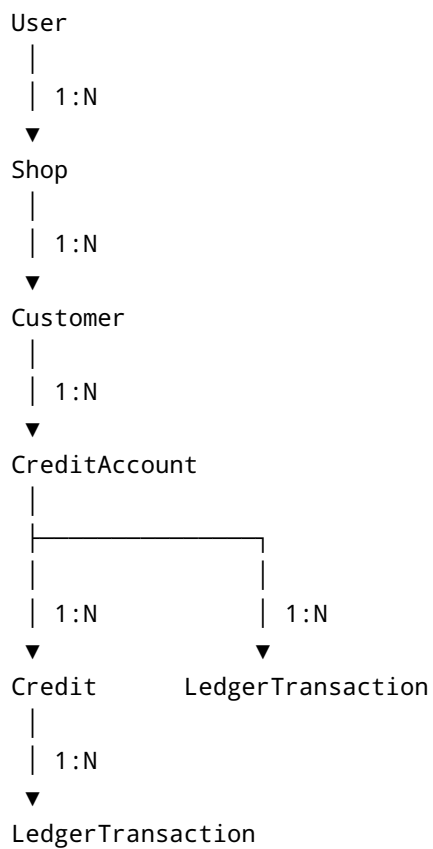
```
outstanding_amount > 0
AND
due_date < current_date
```

This approach prevents inconsistencies between stored status and actual financial conditions.

10. Database Relationships and Constraints

10.1 Relationship Structure

The database relationship can be represented as:



10.2 Ownership Rules

The backend must enforce the following ownership chain:

```
Authenticated User
  ↓
Owns Shop
  ↓
Customer belongs to Shop
  ↓
Credit Account belongs to Customer + Shop
  ↓
Credit belongs to Credit Account
```

This prevents users from accessing or modifying another shop owner's financial information.

10.3 Database Constraint

The following unique constraint must exist:

```
UNIQUE(shop_id, customer_id)
```

This guarantees that a customer can have only one CreditAccount per shop.

11. Credit Creation Workflow

11.1 Credit Creation Request

A shop owner may create a credit containing:

```
Customer:   Ramesh
Amount:     ₹500
Due Date:   20-Aug-2026
Description: Grocery
```

11.2 API Endpoint

```
POST /api/shops/{shopId}/customers/{customerId}/credits
```

11.3 Processing Flow

```
Request
↓
Authenticate JWT
↓
Identify Logged-in User
↓
Verify User Owns Shop
↓
Verify Customer Belongs to Shop
↓
Validate Amount
↓
Validate Due Date
↓
Find or Create CreditAccount
↓
Create Credit
↓
Create CREDIT LedgerTransaction
↓
Update Account Balance
↓
COMMIT
```

The entire operation should execute within a single database transaction.

If any step fails:

```
ROLLBACK
```

The system must not allow a situation where:

```
Credit Created
+
Ledger Transaction Missing
```

The Credit creation operation should therefore be implemented using a Spring `@Transactional` service method.

12. Payment Creation Workflow

12.1 Payment Example

Suppose:

Credit Amount: ₹500
Outstanding Amount: ₹500

The shop owner records:

Payment: ₹200

12.2 API Endpoint

`POST /api/credits/{creditId}/payments`

12.3 Processing Flow

```
Request
↓
Authenticate JWT
↓
Verify Shop Ownership
↓
Load Credit
↓
Lock / Verify Current Balance
↓
Validate Payment > 0
↓
Validate Payment <= Outstanding
↓
Create PAYMENT Transaction
↓
Reduce Credit Outstanding Amount
↓
Update Credit Status
↓
Update Credit Account Balance
↓
COMMIT
```

The payment must be rejected when:

```
payment_amount > outstanding_amount
```

This prevents overpayment.

13. Concurrency Control

13.1 Problem

Payment processing introduces a concurrency problem when multiple requests are received at nearly the same time.

For example:

```
Current Outstanding = ₹500
```

Two requests arrive simultaneously:

```
Request A → Pay ₹400  
Request B → Pay ₹300
```

If both requests read the balance before either transaction updates it, both may see:

```
Outstanding = ₹500
```

Both requests could then incorrectly succeed.

The resulting payments would be:

```
₹400 + ₹300 = ₹700
```

which violates the business rule.

13.2 Pessimistic Locking

The payment processing flow should therefore protect the Credit record while the balance is being updated.

A suitable approach with Spring Data JPA is pessimistic locking.

Conceptually:

```
SELECT credit
FROM credit
WHERE id = ?
FOR UPDATE;
```

The flow becomes:

```
Request A
  ↓
Locks Credit #101
  ↓
Reads Outstanding = ₹500
  ↓
Pays ₹400
  ↓
Outstanding = ₹100
  ↓
COMMIT
  ↓
Releases Lock
```

Then:

```
Request B
  ↓
Obtains Lock
  ↓
Reads Outstanding = ₹100
  ↓
Attempts Payment = ₹300
  ↓
Payment > Outstanding
  ↓
REJECT
```

This ensures that the overpayment rule remains valid even when concurrent payment requests are received.

14. REST API Design

The API should follow a resource-oriented REST design.

14.1 Credit APIs

Create Credit

```
POST /api/shops/{shopId}/customers/{customerId}/credits
```

Get Customer Credits

```
GET /api/shops/{shopId}/customers/{customerId}/credits
```

Get Credit Details

```
GET /api/credits/{creditId}
```

14.2 Payment APIs

Create Payment

```
POST /api/credits/{creditId}/payments
```

Get Payments

```
GET /api/credits/{creditId}/payments
```

14.3 Ledger APIs

Get Credit Transaction History

```
GET /api/credits/{creditId}/transactions
```

Future Shop Ledger

```
GET /api/shops/{shopId}/ledger
```

The shop-level ledger endpoint can later provide a consolidated view of financial transactions for an entire shop.

15. Request and Validation Design

15.1 Create Credit Request

Example:

```
{
  "amount": 500,
  "dueDate": "2026-08-20",
  "description": "Grocery Items"
}
```

The backend must not blindly trust identifiers supplied by the frontend.

In particular, `shopId` should not be treated as trusted merely because it was included in the request URL.

The backend must verify:

```
JWT User
  ↓
Owns Shop?
  ↓
Customer Belongs to Shop?
```

This prevents a user from attempting to access another user's shop.

15.2 Create Payment Request

Example:

```
{
  "amount": 200,
  "paymentDate": "2026-08-13",
  "description": "Cash payment"
}
```

The backend determines the Credit from the URL:

```
POST /api/credits/{creditId}/payments
```

The client should not be allowed to provide arbitrary combinations of:

```
{
  "creditId": 101,
  "customerId": 55,
  "shopId": 7
}
```

and expect the backend to trust those values.

The URL and authenticated ownership checks establish the request context.

16. Balance Management Strategy

There are two primary approaches to calculating outstanding balances.

16.1 Dynamic Calculation

The balance can be calculated from the ledger:

```
SUM(CREDIT)
- SUM(PAYMENT)
+ SUM(ADJUSTMENT)
- SUM(REVERSAL)
```

This provides a ledger-derived balance but may require additional database computation for frequently accessed dashboard information.

16.2 Stored Balance

The system can maintain:

```
Credit.outstandingAmount
CreditAccount.outstandingBalance
```

These values can be updated whenever a financial transaction occurs.

This allows fast access to current balances.

16.3 Recommended Approach

SettleLater should use both approaches.

Stored Balance

For fast dashboard and API queries:

```
Credit.outstandingAmount  
CreditAccount.outstandingBalance
```

Ledger

For transaction history and auditability:

```
LedgerTransaction
```

The critical consistency rule is:

A stored balance must never be updated without creating the corresponding ledger transaction within the same database transaction.

This ensures that the stored balance and financial history remain synchronized.

17. Transaction Management

Financial operations must be atomic.

17.1 Credit Creation

The following operations should occur within one database transaction:

```
Create Credit  
+  
Create CREDIT LedgerTransaction  
+  
Update CreditAccount
```

If any operation fails:

```
ROLLBACK
```

17.2 Payment Creation

The following operations should also occur within one transaction:

```
Create PAYMENT LedgerTransaction  
+  
Update Credit Outstanding  
+
```

```
Update Credit Status
+
Update CreditAccount Balance
```

If any operation fails:

```
ROLLBACK
```

This guarantees that the financial state cannot be partially updated.

18. Implementation Roadmap

The implementation should be completed incrementally rather than introducing every feature at once.

Phase 1 — Core Financial Entities

Implement:

- CreditAccount entity
- Credit entity
- LedgerTransaction entity

Establish their database relationships and constraints.

Phase 2 — Credit Management

Implement:

- Create Credit
- Get Customer Credits
- Get Credit Details
- Get Credit Transaction History

At the end of this phase, the system should be able to create and display credit records and their associated history.

Phase 3 — Payment Management

Implement:

- Record Payment
- Partial Payment
- Full Settlement
- Overpayment Validation

Payment processing should include transaction management and concurrency protection.

Phase 4 — Financial Corrections

Implement:

- Payment Reversal
- Credit Adjustment
- Adjustment Reversal
- Audit History

This phase establishes the immutable financial-record model.

Phase 5 — Account Intelligence

Implement:

- Overdue Calculation
- Search
- Filters
- Dashboard Statistics

Overdue status should be calculated using the outstanding balance and due date.

Phase 6 — Notifications

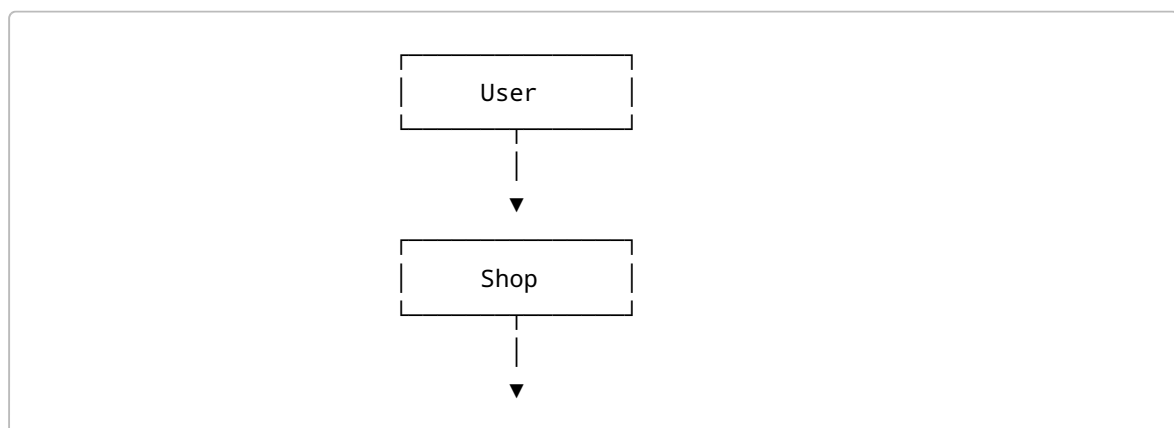
Implement:

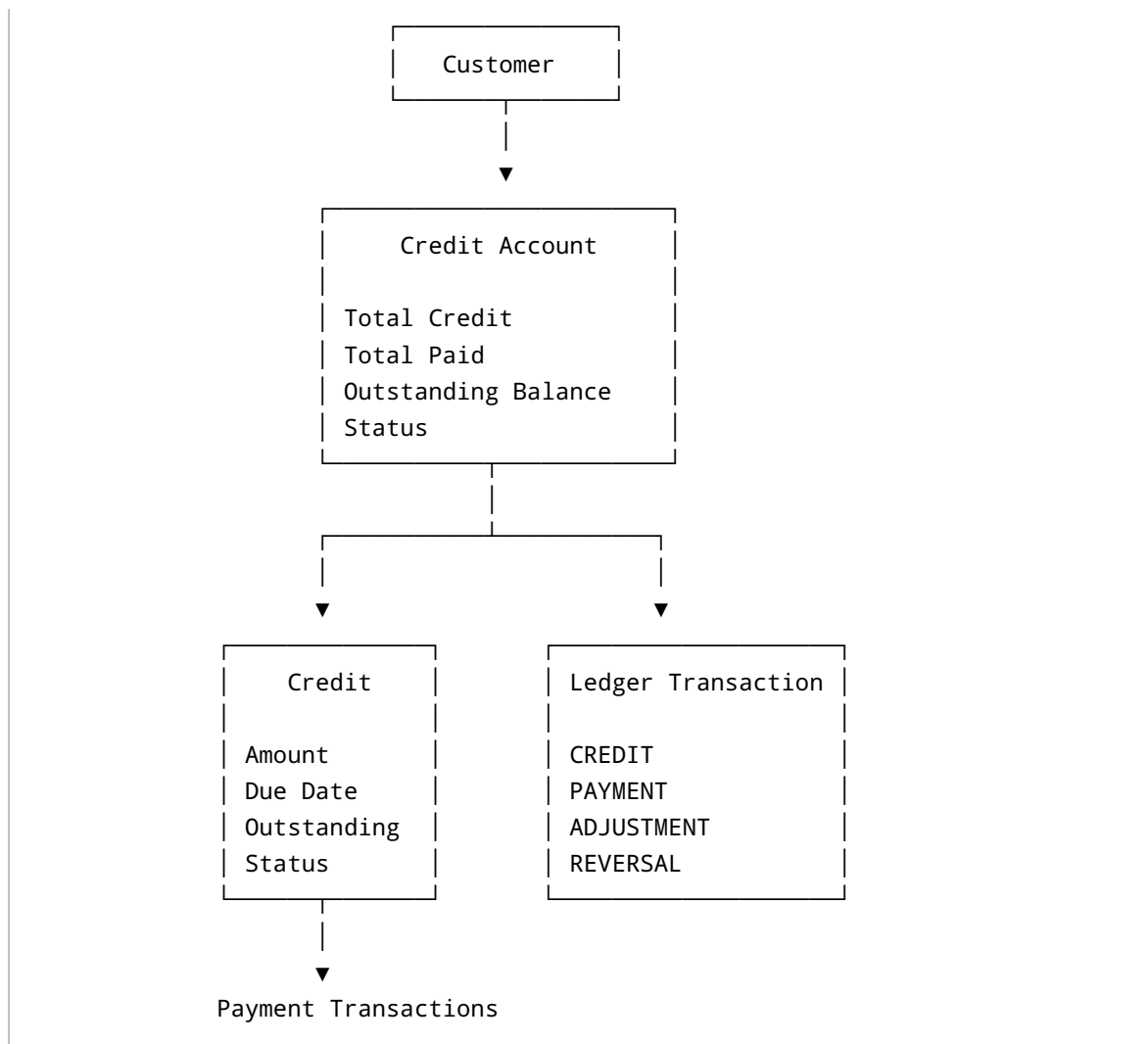
- Payment Notifications
- Due-Date Reminders
- SMS Notifications
- Email Notifications

Notifications should be added after the core financial model is stable.

19. Final Architecture

The final financial architecture is:





The architecture provides a clear separation between:

- The customer-shop relationship.
- The customer's account balance.
- Individual credit obligations.
- Individual financial transactions.

20. Design Principles and Future Extensibility

The SettleLater financial module is designed around several important engineering principles.

20.1 Financial Records as Ledger Data

Financial history should not be treated like ordinary CRUD data.

Transactions should remain traceable and corrections should be represented through additional ledger events rather than destructive edits.

20.2 Strong Ownership Validation

Every protected financial operation must verify the complete ownership chain:

```
Authenticated User
  ↓
Shop Ownership
  ↓
Customer-Shop Relationship
  ↓
Credit Account
  ↓
Credit
```

This provides shop-level data isolation.

20.3 Atomic Financial Operations

Financial operations must be executed atomically using database transactions.

A credit, its ledger transaction, and its account balance update should either all succeed or all fail.

20.4 Concurrency Safety

Payment processing must account for concurrent requests.

Pessimistic locking provides a reliable mechanism for protecting the current Credit balance during payment processing.

20.5 Stored Balance + Ledger

The system maintains stored balances for performance while retaining a ledger for auditability.

The ledger provides the historical explanation for the stored financial state.

20.6 Extensibility

The architecture is intentionally designed so that future features can be added without redesigning the financial model.

Potential future capabilities include:

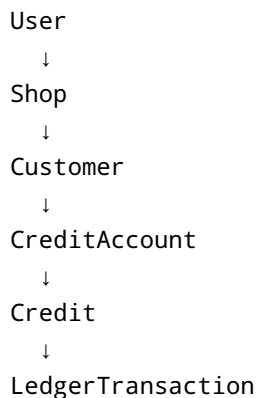
- Payment reminders.
- Due-date notifications.
- SMS notifications.
- Email notifications.
- Customer login.

- Detailed payment history.
 - Shop-wide ledger reporting.
 - Dashboard analytics.
 - Dispute handling.
 - Financial adjustments and reversals.
-

Conclusion

The SettleLater financial architecture provides a structured and reliable foundation for digital credit management.

The core model is based on the relationship:



A customer can have multiple individual credits under a shop, while the CreditAccount provides an aggregated view of the customer's outstanding balance.

The use of an immutable ledger ensures that financial history remains traceable. Payments and credit corrections are handled through transaction records such as PAYMENT, ADJUSTMENT, and REVERSAL rather than destructive edits.

The architecture also incorporates:

- Database-level uniqueness constraints.
- Shop ownership validation.
- Transactional financial operations.
- Overpayment protection.
- Pessimistic locking for concurrent payments.
- Stored balances for performance.
- Ledger transactions for auditability.

This provides a clean MVP architecture while leaving sufficient flexibility for future features such as reminders, notifications, analytics, customer access, and dispute handling.